PROJECT <A>

2026년 미니 프로젝트 종합본

목차

- 기여자 소개
- 개발 일정
- 아키텍처
- 제품 개발
- AI 모델
- 문제 해결
- 참고 자료

한 팀, 네 가지 전문성

Backend · AI · Frontend · Infra가 하나의 OCR 서비스로 연결됩니다.

01 BACKEND

김영민

- Spring Boot 기반 API 개발 및 React–FastAPI OCR 연동
- JWT 기반 사용자 인증 구현
- 문서 업로드와 OCR결과 반환의 처리 흐름 구성
- MySQL 연동 및 데이터 저장 구조 구현
- Mockito 단위 테스트와 테스트 계획·결과서 작성
- 프로젝트 계획서 화면 정의서 작성

02 PM · INFRA

최건우

- GitHub 조직·리포지토리 및 프로젝트 리소스 관리
- Docker 이미지와 컨테이너 상태 점검 체계 운영
- .env 기반 조직 단위 환경 변수와 배포 설정
- OA 자료 배포 및 라이브 서비스 운영
- Web UI 구성

Backend → AI → Frontend → Infra 운영까지, 하나의 End-to-End OCR 경험을 함께 구현했습니다.

한 팀, 네 가지 전문성

Backend · AI · Frontend · Infra가 하나의 OCR 서비스로 연결됩니다.

03 AI MODEL

최민기

- FastAPI OCR 서버 개발 및 AI 모델 연동
- OCR API 단위 테스트와 pytest 테스트 코드 작성

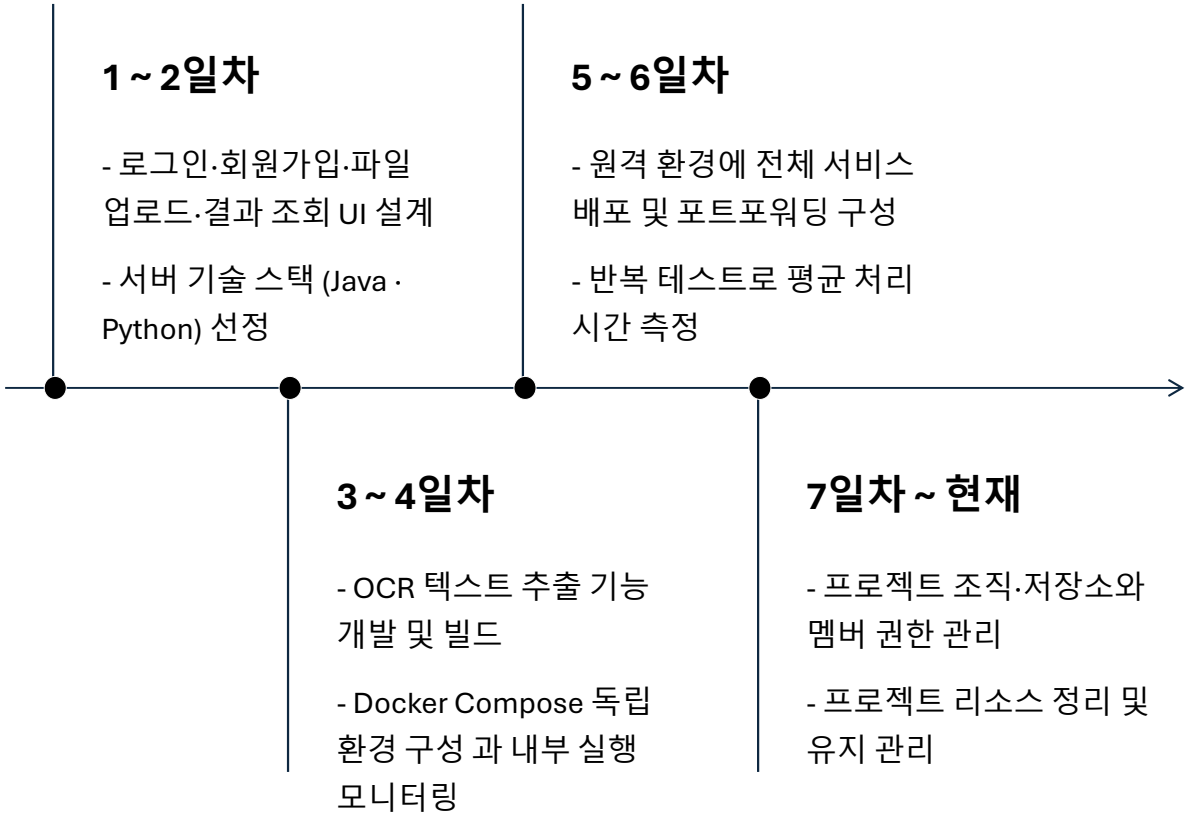
04 FRONTEND

윤승진

- React/Vite 기반 OCR 문서 관리 화면 구현
- API 연동 및 사용자 이벤트 처리 로직 개발

Backend → AI → Frontend → Infra 운영까지, 하나의 End-to-End OCR 경험을 함께 구현했습니다.

개발 일정

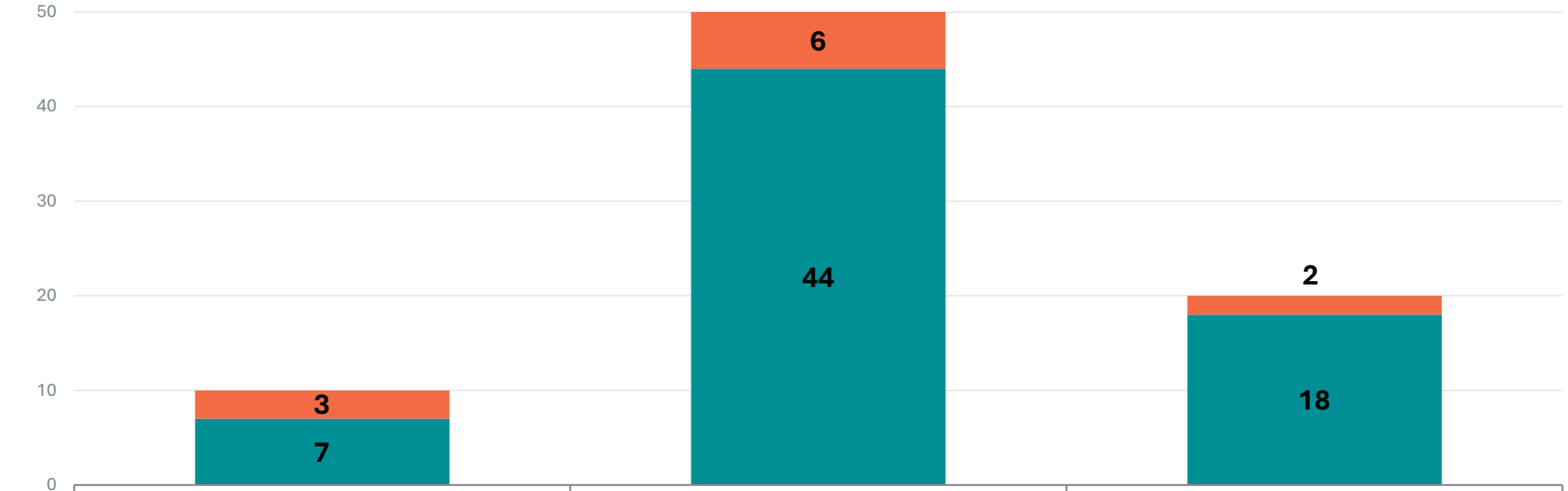

1 ~ 2일차
- 로그인·회원가입·파일 업로드·결과 조회 UI 설계
- 서버 기술 스택 (Java · Python) 선정
3 ~ 4일차
- OCR 텍스트 추출 기능 개발 및 빌드
- Docker Compose 독립 환경 구성 과 내부 실행 모니터링
5 ~ 6일차
- 원격 환경에 전체 서비스 배포 및 포트포워딩 구성
- 반복 테스트로 평균 처리 시간 측정
7일차 ~ 현재
- 프로젝트 조직·저장소와 멤버 권한 관리
- 프로젝트 리소스 정리 및 유지 관리

개발 브랜치 전략


v1.0
(배포 태그)
v1.0.1
(배포 태그)
시간
main
배포 이력
main으로 병합
(v1.0 출시)
main으로 병합
(v1.0.1 출시)
hotfix
긴급 수정
긴급 수정
커밋 1
develop
개발 통합
develop으로 병합
(변경 사항 반영)
develop으로 병합
(수정 사항 반영)
feature
기능 개발
기능 개발
커밋 1
기능 개발
커밋 2
기능 개발
커밋 3

PR 80건 중 69건 승인

리포지토리별 **승인**·**취소** 현황 | server 50건 · web 20건 · infra 10건



	Infra	Server	Web
취소	3	6	2
승인	7	44	18

PR 생성부터 협업 공유까지 자동화

- **GitHub Actions**가 사용자 국적에 맞는 언어별 PR 템플릿을 적용해 개발 현황을 빠르게 공유하도록 지원

AUTOMATION FLOW

01 사용자 정보 확인

GitHub API로 국적 정보 조회

02 언어별 템플릿 적용

국적에 맞는 PR 템플릿 자동 선택

03 변경 요약 확장

Commit SHA-256 기반 주요 변경 사항 반영 예정

국적 확인 → 템플릿 적용 → 변경 요약

PR 작성 규칙을 자동화해 협업 공유 속도를 높임

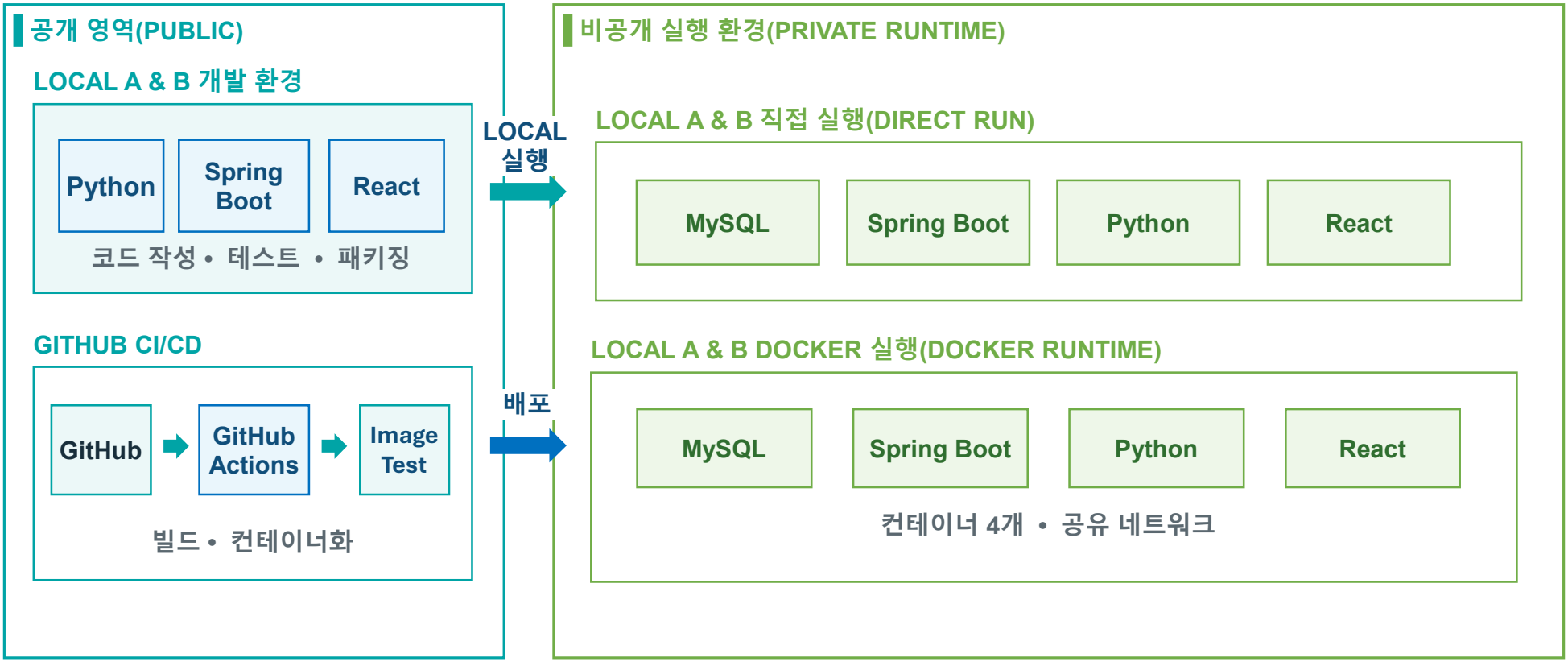
PR TEMPLATE | 자동화 적용 화면



아키텍처 & 인프라

발표자 : 최건우

아키텍처



개발 → 빌드 → 컨테이너화 → 배포 → 실행

조직 단위 배포 환경 변수 값 관리

01 • 리포지토리 변수 정의

```
# Environment variables for the server services
WEB_FRONTEND_PORT=YOUR_WEB_FRONTEND_PORT
GO_SERVER_PORT=YOUR_GO_SERVER_PORT

# PostgreSQL environment variables for the project
POSTGRES_DB=YOUR_DATABASE_NAME
POSTGRES_USER=YOUR_DATABASE_USER
POSTGRES_PASSWORD=YOUR_DATABASE_PASSWORD
POSTGRES_HOST=YOUR_DATABASE_HOST
POSTGRES_PORT=YOUR_DATABASE_PORT

# MySQL environment variables for the project
MYSQL_DATABASE=YOUR_DATABASE_NAME
MYSQL_USER=YOUR_DATABASE_USER
MYSQL_PASSWORD=YOUR_DATABASE_PASSWORD
MYSQL_ROOT_PASSWORD=YOUR_DATABASE_PASSWORD
MYSQL_HOST=YOUR_DATABASE_HOST
MYSQL_PORT=YOUR_DATABASE_PORT
```

```
# Python server environment variables
PYTHON_SERVER_PORT=YOUR_PYTHON_SERVER_PORT
APP_NAME=YOUR_APP_NAME
APP_VERSION=YOUR_APP_VERSION
APP_ENV=YOUR_APP_ENV
DEBUG=YOUR_DEBUG
AI_PROVIDER=YOUR_AI_PROVIDER

OPENAI_API_KEY=YOUR_OPENAI_API_KEY
OPENAI_MODEL=YOUR_OPENAI_MODEL

PDF_BACKEND=YOUR_PDF_BACKEND
DEFAULT_OCR_ENGINE=YOUR_DEFAULT_OCR_ENGINE

AI_TIMEOUT_SECONDS=YOUR_AI_TIMEOUT_SECONDS
AI_MAX_OUTPUT_TOKENS=YOUR_AI_MAX_OUTPUT_TOKENS

NETWORK_ENV=YOUR_NETWORK_ENV
ENGINE=YOUR_ENGINE
```

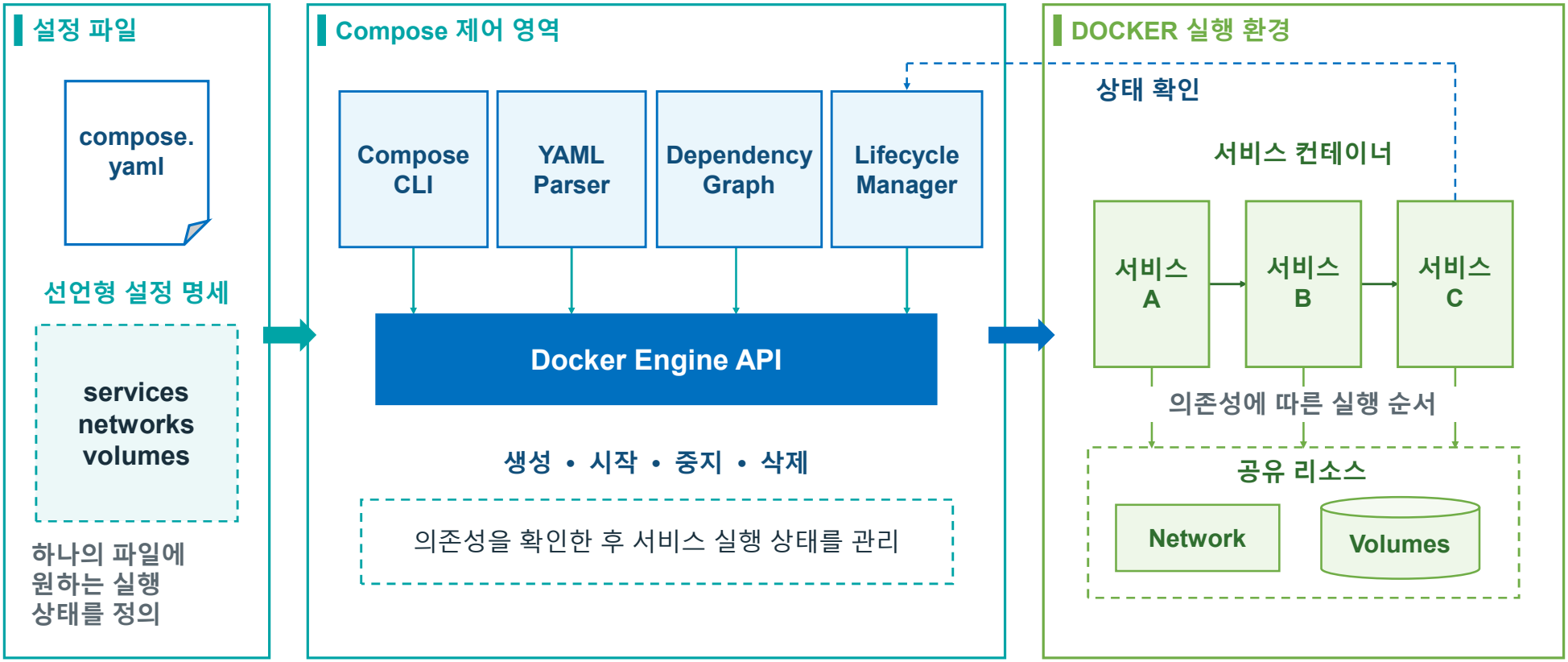
→

02 • 조직 시크릿 등록



변수 정의 → 조직 단위 등록 → 배포 환경에서 일관되게 사용

아키텍처



설정 선언 → 설정 해석 → 의존성 확인 → 실행 제어 → 서비스 실행

Docker Compose 기반 서비스 실행

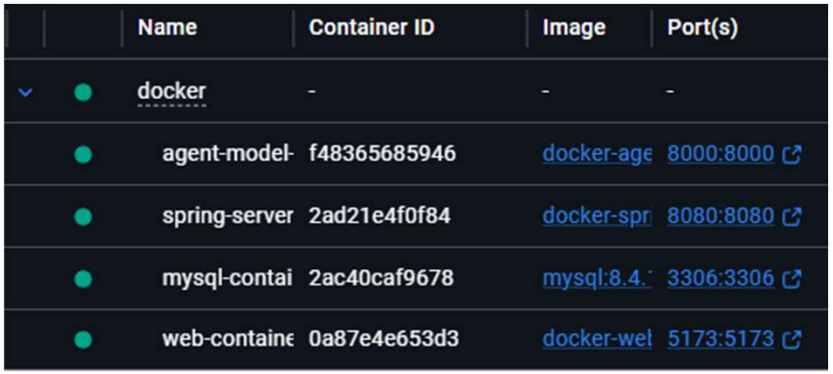
01 • 실행 구성 정의

```yaml
# MySQL Database Service
▷Run Service
ocr-mysql:
  container_name: mysql-container
  image: mysql:8.4.11-oraclelinux9
  shm_size: '256mb'
  restart: always
  volumes:
    - ../db/schema.sql:/docker-entrypoint-initdb.d/schema.sql:ro
    - mysql-data:/var/lib/mysql
  environment:
    MYSQL_HOST: ${MYSQL_HOST}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  ports:
    - "${MYSQL_PORT}:3306"
  healthcheck:
    test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -p$$MYSQL_ROOT_PASSWORD --silent"]
    interval: 10s
    timeout: 5s
    retries: 20

volumes:
  mysql-data:
    driver: local
```

→

02 • 컨테이너 상태 확인



Compose 정의 → 서비스 컨테이너 실행

React Web

발표자 : 윤승진

Components는 공통 기능, Pages는 사용자 흐름 담당

재사용·인증 제어와 화면별 입력·API 처리를 분리한 구조

SHARED LAYER

Components/

여러 화면이 공유하는 UI와 인증 로직

protected-route.jsx

토큰 검증 · 보호 화면 접근 제어

공통 인증 흐름을 화면 진입 전에 처리

feature-header.jsx

브랜드 헤더 · 로그아웃 이벤트

여러 화면에서 동일한 UI와 동작 재사용

ROUTE LAYER

Pages/

URL 단위 실제 화면과 사용자 이벤트

signin.jsx · register.jsx

로그인·회원가입 입력 · API 호출

feature.jsx · upload.jsx

문서 목록 · PDF 업로드 · 엔진 선택

document-detail.jsx

폴링 · 결과 표시 · 재처리 · 삭제 · 내보내기

Components로 횡단 기능을 모으고, Pages는 화면별 작업에 집중

App 라우팅이 페이지 흐름을 잇고, 공통 컴포넌트가 보호 화면을 지탱한다

로그인 → 문서 목록 → PDF 업로드까지, 화면 전환과 API 호출을 한 흐름으로 연결

ROUTE FLOW | Pages/



보호 화면에 공통 적용

SHARED LAYER | Components/

`protected-route.jsx`
보호 화면 진입 전 인증 확인

`feature-header.jsx`
공통 헤더 · 로그아웃 처리

핵심 로직 | App 라우팅 → Pages 업무 처리 → Components 인증·UI 재사용

Spring boot 서버

발표자 : 김영민

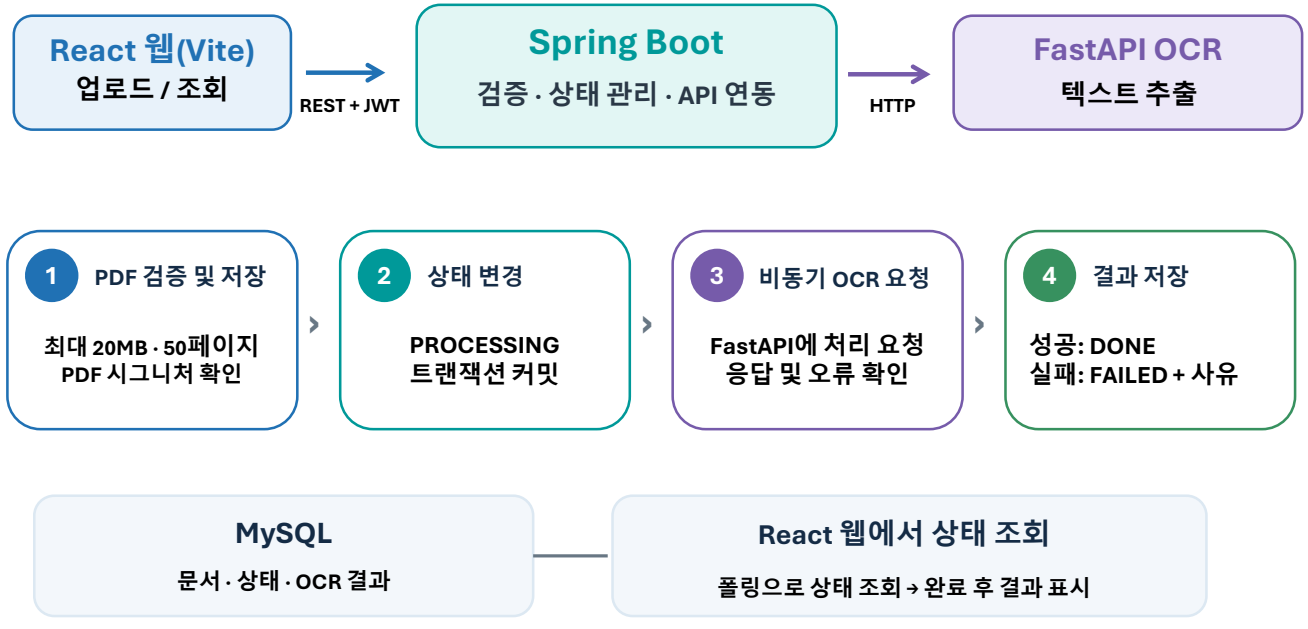
Spring Boot 서버 역할 및 처리 흐름

01 JWT 인증

02 PDF 검증·저장

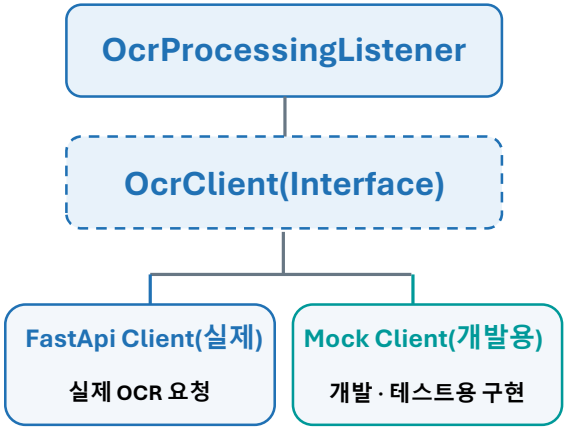
03 처리 상태 관리

04 결과 저장



설계 포인트 및 향후 확장 방향

01 교체 가능한 OCR 클라이언트



- 설정으로 구현체 전환(app.ocr.client)

02 단위 테스트 및 예외 처리

161개
단위 테스트 케이스

DB · 네트워크없이 (Mockito 기반 검증)

- 파일 경계값
- 인증
- OCR 흐름
- 오류 응답

- 타임아웃 · 오류 매핑

03 향후 확장: 벡터 DB

기존 구조를 유지하며 단계 추가



- 향후 RAG 기능으로 확장

AI Model

발표자 : 최민기

처리 속도와 정밀도에 따라 OCR 모델을 선택

공통 기준 오픈소스 라이선스 MIT · Apache-2.0 | 지원 언어 English · Korean

SPEED / VOLUME

Tesseract

4.0 버전부터 LSTM 기반

- 상대적으로 가볍고 CPU에서 빠름
- 대량 문서를 신속하게 처리할 때 유리

PRECISION / COMPLEX INPUT

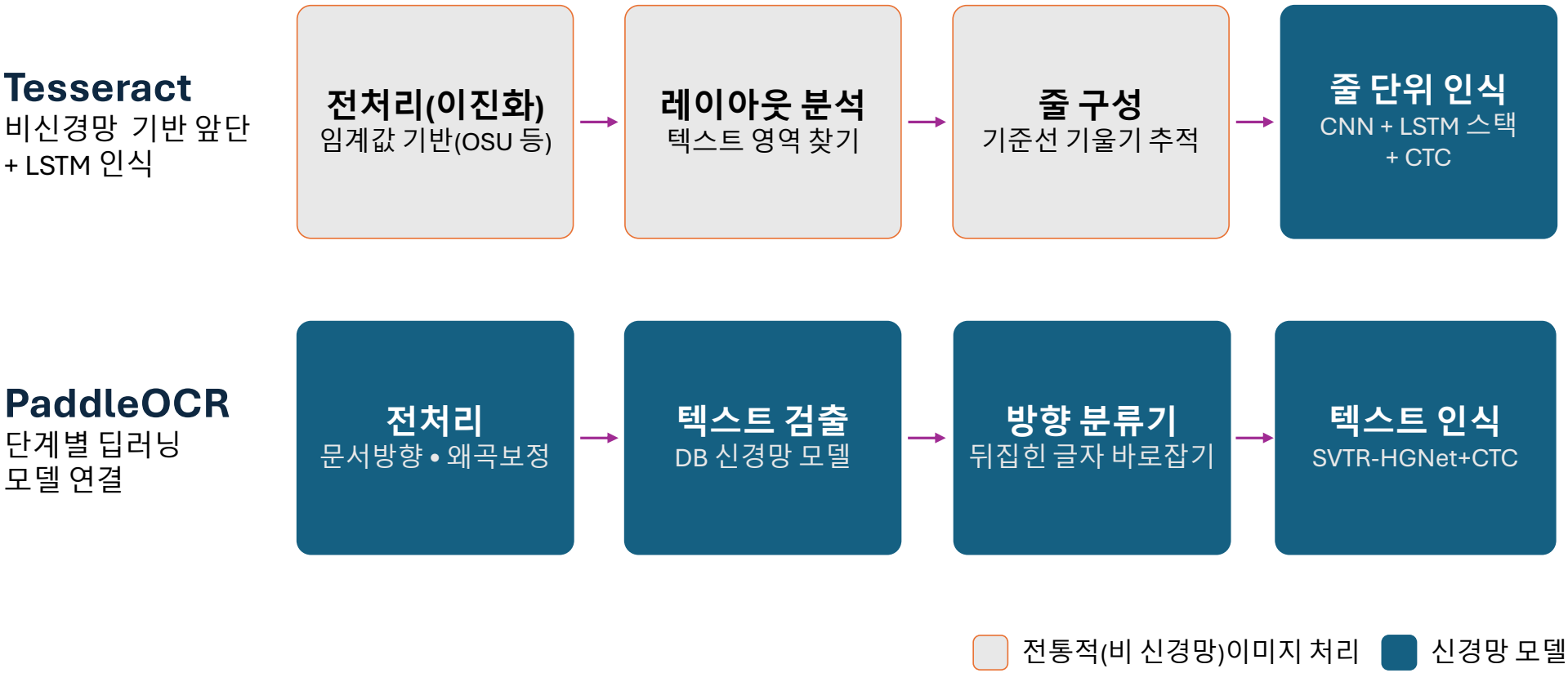
PaddleOCR

전 과정 딥러닝 기반

- CPU 환경 쓰레드 활용, GPU 가속 가능
- 복잡한 이미지를 정밀하게 인식할 때 유리

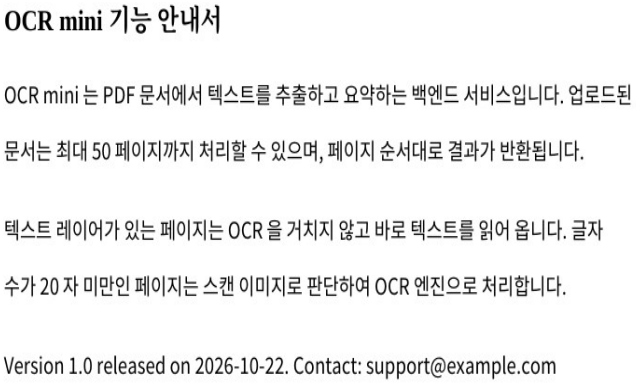
AI 모델 로직

입력 이미지에서 텍스트가 나오기까지의 처리 흐름



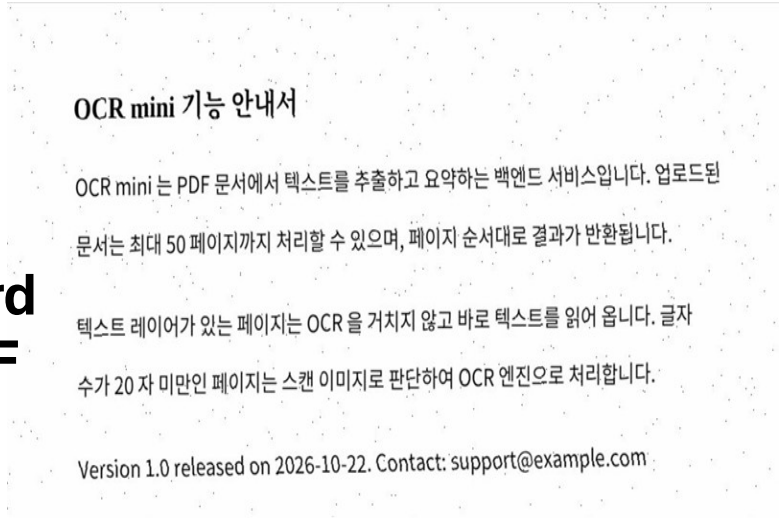
AI 모델 벤치마킹

Normal PDF



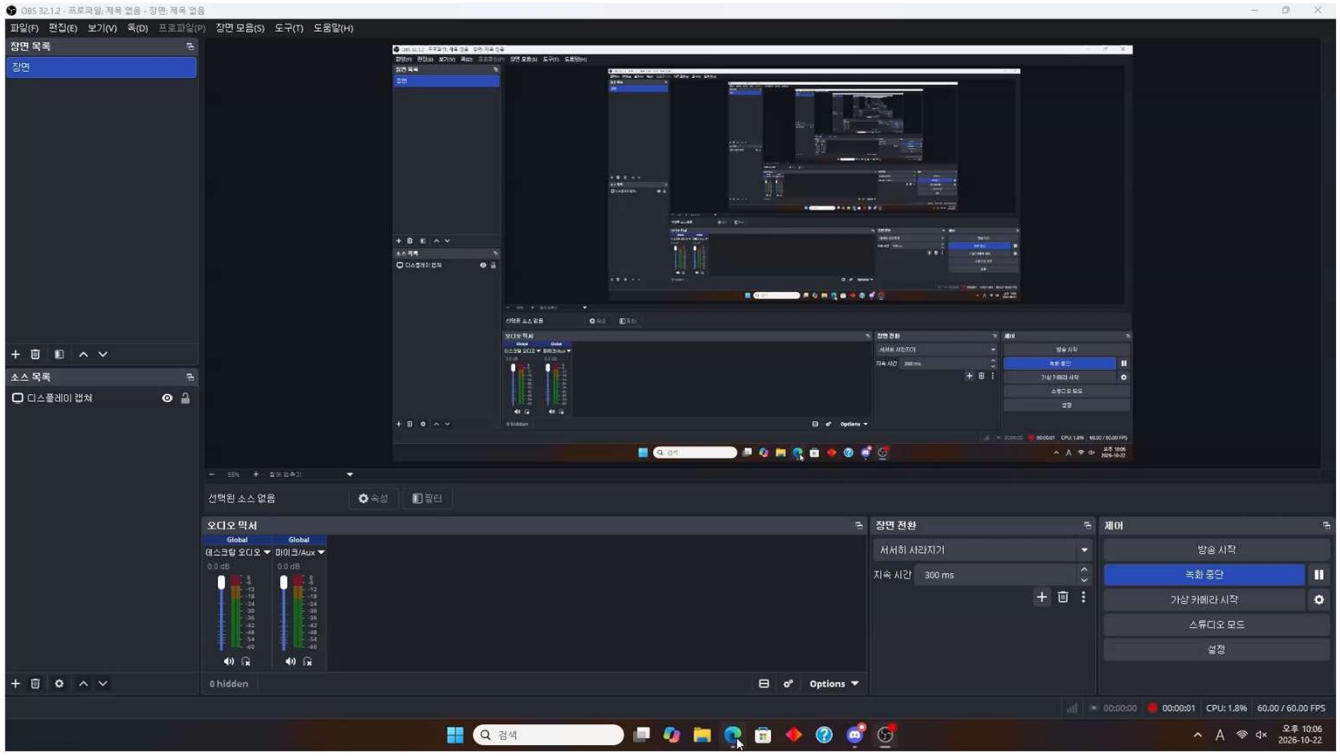
	CER(%)	정확도(%)	처리 시간 (Ms)
Tesseract	34	66	7491
Paddle	4	96	32455

Hard PDF



	CER(%)	정확도(%)	처리 시간(Ms)
Tesseract	38	62	7016
Paddle	10	90	16881

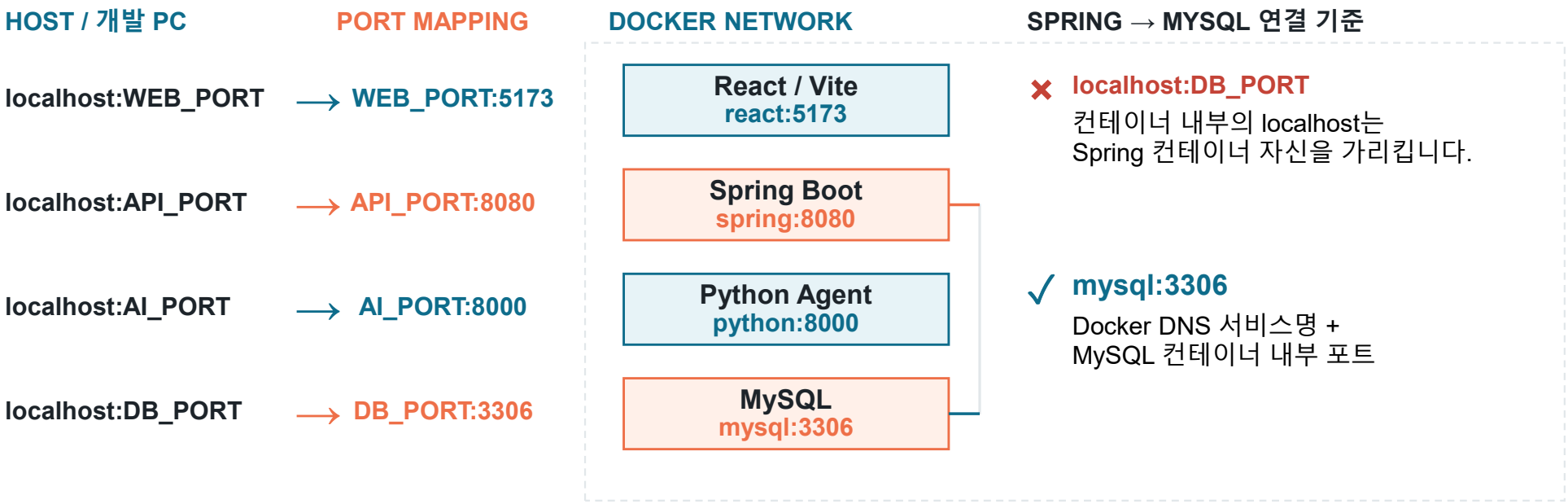
AI 모델 데모 영상


파일(F) 편집(E) 보기(V) 독(D) 프로파일(P) 장면 모음(S) 도구(T) 도움말(H)
장면 목록
장면
소스 목록
디스플레이 캡처
선택된 소스 없음
속성
필터
오디오 믹서
Global
Global
데스크탑 오디오
마이크/Aux
0.0 dB
0.0 dB
0 hidden
Options
장면 전환
서서히 사라지기
지속 시간
300 ms
제어
방송 시작
녹화 중단
가상 카메라 시작
스튜디오 모드
설정
00:00:00
00:00:01
CPU: 1.8%
60.00 / 60.00 FPS
검색
오후 10:06
2026-10-22

문제 해결

Container Port Mapping 문제 해결

HOST_PORT는 외부 공개용, 서비스 간 통신은 Docker 서비스명 + CONTAINER_PORT를 사용합니다.



원인 Host 공개 포트와 Container 내부 포트를 동일한 참조 기준으로 사용

해결 Spring: DB_HOST=mysql · DB_PORT=3306
외부 접속만 localhost:DB_PORT

향후 개발 계획

요약

프로젝트 종합

우리는 전체 개발, 배포, OCR 워크플로우를 연결했습니다.
이제 운영 품질과 확장성을 향상시킬 때입니다.

프로젝트 요약

01 협업 및 배포

분기 전략을 GitHub Actions와 연동하여 코드 변경부터 빌드, 테스트, 컨테이너화까지 흐름을 구축했습니다.

02 프로덕션 준비 아키텍처

Python, Spring Boot, React, MySQL 모두 직접 실행과 Docker 런타임을 지원하며, 컴포즈 기반 라이프사이클을 가지고 있습니다.

03 End-to-End OCR 애플리케이션

사용자 요청과 API 검증을 통해 에이전트 전처리, 테서랙트 추론, MySQL 저장소를 통해 핵심 흐름을 구현했습니다.

예상 로드맵

1단계 작전 안정성

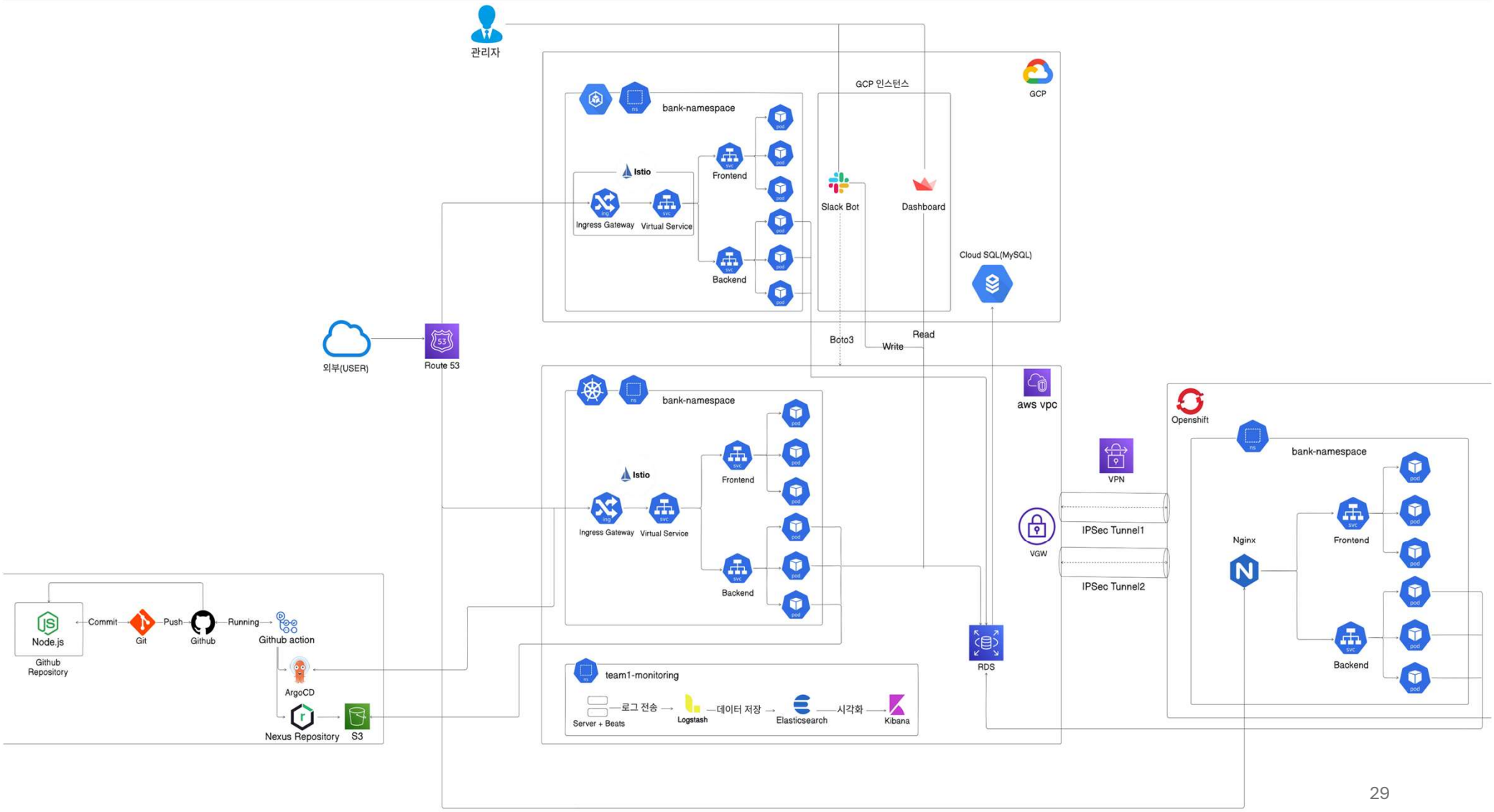
통합 테스트 확장 · 오류 처리 개선 · 성능 기준선 정의

2단계 자동화 및 관측 가능성

CI/CD 품질 게이트 강화 · 로그 및 메트릭 기반 가시성 개선

3단계 규모 및 최적화

확장 가능한 인프라 평가 · OCR/AI 모델 개선 · 서비스 확장


관리자
GCP
GCP 인스턴스
bank-namespace
Istio
Ingress Gateway
Virtual Service
Frontend
Backend
Slack Bot
Dashboard
Cloud SQL(MySQL)
Boto3
Write
Read
외부(USER)
Route 53
aws vpc
VPN
IPSec Tunnel1
IPSec Tunnel2
VGW
RDS
Openshift
Nginx
team1-monitoring
Server + Beats
로그 전송
Logstash
데이터 저장
Elasticsearch
시각화
Kibana
Node.js
Github Repository
Commit
Git
Push
Github
Running
Github action
ArgoCD
Nexus Repository
S3

관련 링크



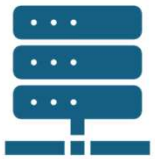
팀원 프로필

geonwoo8873 (GEONWOO CHOI)

GreyColor10 (Mingi Choi)

Kymin9118

tmdwls1226



프로젝트 저장소

geonwoo8873/global-academy-project [Private]

Org-global-academy-project



참고 자료

geonwoo8873/custom-repo-template

Hugging Face – The AI community building the future.

Vite | Next Generation Frontend Tooling

W3Schools Online Web Tutorials

wingetCollections

Live Demo & Experience



- **체험 환경 운영:**
  - 10/23/2026 ~ 10/24/2026
  - ID : asd@asd / PS : *abc1234
- **서비스 제공 기능:**
  - PDFIUM
  - TESSERACT
  - PADDLE
- **Demo GitHub Repository:**
  - global-academy-project/mini-project-demo
    [상태 : 마이그레이션 준비중]

THANK YOU